

01 / WARUM VEREINFACHEN?

Tag 8: Karnaugh-Diagramme

An Tag 7 hast du aus jeder Einerzeile einen Minterm gebaut. Das funktioniert immer, wird aber schnell lang. Heute suchst du eine kürzere SOP, die bei allen festgelegten Eingängen denselben Ausgang liefert.

Der bekannte Rechenschritt hinter dem Bild

$$Y = A\overline{B} + AB = A(\overline{B} + B) = A \cdot 1 = A$$

B kann 0 oder 1 sein; in beiden Fällen ist Y genau dann 1, wenn A=1. Deshalb darf B aus der Gleichung verschwinden. Eine Karnaugh-Map, auch KV-Diagramm genannt, macht solche Paare sichtbar.

Eine K-Map ist eine umgeordnete Wahrheitstabelle. Jede Zelle steht für eine Eingangskombination und enthält deren Ausgangswert.

Drei Eingänge: unsere feste Anordnung

Wir schreiben A an die Zeilen und BC an die Spalten. A bleibt das höchstwertige Bit. Die Zahlen unten sind zunächst **Indizes**, keine Ausgangswerte:

A / BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

Zum Beispiel: Zeile A=1, Spalte BC=10 ergibt ABC=110, also Index $4 + 2 = 6$. **Im Buch stehen beim Drei-Variablen-Beispiel AB oben und C links.** Das ist ebenfalls richtig. Hier verwenden wir durchgehend A links und BC oben. Nicht zwischen den Anordnungen wechseln, ohne die Indizes neu zuzuordnen.

Warum 00, 01, 11, 10?

Das ist die **Gray-Code-Reihenfolge**. Zwischen benachbarten Spalten ändert sich genau ein Bit. Bei 01 zu 11 ändert sich nur B. Bei der normalen Reihenfolge 01 zu 10 würden sich zwei Bits ändern; solche Felder lassen sich nicht zu einem Zweierblock zusammenfassen.

Das Lernziel

Du überträgst Tabellenwerte sicher in eine K-Map, bildest gültige Gruppen, liest daraus Terme ab und prüfst die gekürzte Gleichung. Das Python-Verfahren kommt erst danach.

02 / ZWEI FELDER ZUSAMMENFASSEN

Was gleich bleibt, bleibt im Term

Gegeben ist $Y = \Sigma m(4, 5)$. Nur die Indizes 4 und 5 bekommen eine 1; alle anderen Zellen bekommen eine 0.

A / BC	00	01	11	10
0	0	0	0	0
1	1	1	0	0

Schritt 1: Die Eingangsbits aufschreiben

Index	A	B	C
4	1	0	0
5	1	0	1

Schritt 2: Konstante und wechselnde Bits unterscheiden

A ist in beiden Zellen 1: A bleibt. B ist in beiden Zellen 0: $\overline{B}$ bleibt. C wechselt zwischen 0 und 1: C entfällt.

$$Y = A\overline{B}$$

Schritt 3: Durch Rechnung begründen

$$A\overline{B}\overline{C} + A\overline{B}C = A\overline{B}(\overline{C} + C) = A\overline{B}$$

Ein Kreis oder Rechteck bedeutet also nicht „irgendwie zusammenzählen“. Die Gruppe fasst genau die Kombinationen zusammen, für die eine Variable keine Rolle mehr spielt.

Gruppengröße und Anzahl der Literale

Bei drei Eingängen enthält eine Einzelzelle drei Literale. Eine Zweiergruppe enthält zwei, eine Vierergruppe eines. Sind alle acht Zellen 1, gilt $Y = 1$.

Für eine SOP-Gruppe: Ein konstantes Bit 1 ergibt die normale Variable, ein konstantes Bit 0 die negierte Variable. Alles, was innerhalb der Gruppe wechselt, wird weggelassen.

03 / GRUPPENRICHTLINIEN

Ränder gehören zusammen

1. Für SOP gruppierst du Einsen. Eine Gruppe darf keine festgelegte Null enthalten.
2. Gruppen sind Rechtecke mit 1, 2 oder 4 Feldern je Kantenrichtung. Erlaubte Größen sind 1, 2, 4, 8 oder 16; keine Dreier- oder Sechsergruppen.
3. Nachbarschaft gilt waagrecht und senkrecht, nicht diagonal.
4. Linker und rechter Rand sind benachbart; ebenso oberer und unterer Rand.
5. Gruppen dürfen sich überlappen. Jede benötigte 1 muss mindestens einmal abgedeckt sein.
6. Suche möglichst große Gruppen und anschließend eine Abdeckung mit möglichst wenigen Termen. Bei gleicher Termzahl bevorzuge weniger Literale.

Ein Randbeispiel mit Überlappung

$Y = \Sigma m(0, 2, 4, 5, 6)$:

A / BC	00	01	11	10
0	1	0	0	1
1	1	1	0	1

Die beiden durchgezogenen Randstreifen bilden **zusammen eine Vierergruppe** über den linken und rechten Rand: Indizes 0, 2, 4, 6. Dort bleibt nur $C=0$ konstant. Ihr Term ist $\overline{C}$. Die gestrichelte Zweiergruppe enthält 4 und 5. Ihr Term ist $A\overline{B}$. Index 4 liegt in beiden Gruppen; das ist erlaubt.

$$Y = \overline{C} + A\overline{B}$$

Wie prüfe ich die Gruppierung?

Schreibe die Indizes jeder Gruppe auf. Kontrolliere, dass alle enthaltenen Zellen 1 sind und jede 1 irgendwo abgedeckt ist. Prüfe danach die Gleichung auch an den Nullzeilen: Eine zu große Gruppe könnte unbemerkt eine Null einschließen.

04 / VIER EINGÄNGE

Die 4-mal-4-Karte

Bei vier Eingängen stehen AB an den Zeilen und CD an den Spalten. **Beide** Achsen benutzen 00, 01, 11, 10. Die Indizes lauten:

AB / CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Beispiel: AB=10 und CD=11 ergibt ABCD=1011, also Index $8 + 2 + 1 = 11$. Eine Wahrheitstabelle in normaler Binärreihenfolge darfst du deshalb nicht einfach zeilenweise in die Karte kopieren.

Die vier Ecken sind eine Gruppe

Für $Y = \Sigma m(0, 2, 8, 10)$ sieht die Karte so aus:

AB / CD	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

Die vier markierten Ecken bilden über beide Ränder **eine Vierergruppe**. A und C wechseln. B=0 und D=0 bleiben konstant:

$$Y = \overline{B} \overline{D}$$

Die Karte ist am Rand nicht zu Ende. Prüfe auch Gruppen über links/rechts, oben/unten und über alle vier Ecken.

05 / BEWUSST AUSWÄHLEN

Primimplikanten und Don't-Cares

Was ist ein Primimplikant?

Ein **Implikant** ist hier ein gültiger UND-Term, dessen Gruppe nur erlaubte Einsen abdeckt. Ein **Primimplikant** entspricht einer Gruppe, die sich nicht mehr zu einer größeren gültigen Gruppe erweitern lässt. Das heißt nicht, dass sie die größte Gruppe der gesamten Karte sein muss. Ein **essentieller Primimplikant** deckt mindestens eine benötigte 1 ab, die kein anderer Primimplikant abdeckt. Diesen Term musst du in eine Abdeckung durch Primimplikanten aufnehmen. **Wichtig:** Nicht automatisch alle Primimplikanten übernehmen. Wähle die nötigen aus, sodass alle Einsen abgedeckt sind. Mehrere gleich gute Minimalformen können existieren.

Was bedeutet X?

Ein **Don't-Care** ist eine Eingangskombination, für die die Spezifikation den Ausgang nicht festlegt. Du darfst sie beim Vereinfachen als 0 oder 1 behandeln. Das X ist hier keine unbekannte Messspannung und kein zusätzlicher Logikzustand, den die Schaltung erzeugen soll. Gegeben: Einerindizes $\{1, 3\}$, Don't-Cares $d = \{5, 7\}$. Alle anderen Indizes sind fest 0.

A / BC	00	01	11	10
0	0	1	1	0
1	0	X	X	0

Wir benutzen beide X, um eine Vierergruppe zu bilden. A und B wechseln, C bleibt 1: $Y = C$. Ohne die X wäre für die beiden Einsen nur $\overline{A}C$ möglich.

X darf helfen, muss aber nicht abgedeckt werden. Jede festgelegte 1 muss abgedeckt werden. Eine festgelegte 0 darf niemals in eine SOP-Gruppe geraten.

Das ist eine Entscheidung der Spezifikation

Wenn eine angeblich unbenutzte Eingabe später doch vorkommt, liefert die gebaute Schaltung trotzdem einen Wert. Nur darfst du dafür keinen bestimmten Wert versprechen. Verwende X daher nicht, um eine unbequeme Null aus der Aufgabe zu entfernen.

06 / VERGLEICH MIT TAG 7

POS und die vollständige Kontrolle

SOP: Einsen gruppieren

Aus jeder Einsengruppe wird ein UND-Term. Konstante 0 negieren, konstante 1 unverändert lassen. Danach alle Gruppenterme mit ODER verbinden.

POS: Nullen gruppieren

Du kannst stattdessen Nullgruppen bilden. Aus jeder wird eine ODER-Klammer. Damit sie für die Nullgruppe 0 ergibt, gilt die umgekehrte Regel: konstante 0 unverändert, konstante 1 negieren. Danach die Klammern mit UND verbinden. **Beispiel:** Y ist bei Index 0 und 1 gleich 0, sonst überall 1. Die beiden Nullzellen haben $A=0$ und $B=0$ gemeinsam; C wechselt. Daraus folgt die POS $Y = (A + B)$.

Die Schritte für jede Aufgabe

1. Variablenreihenfolge und Gray-Code-Beschriftung festlegen.
2. Alle 1-, 0- und gegebenenfalls X-Werte an den richtigen Index setzen.
3. Gültige Gruppen markieren und ihre Zellindizes notieren.
4. Für jede Gruppe konstante Bits ablesen und den Term bilden.
5. Die Terme passend zu SOP bzw. POS verbinden.
6. Für alle festgelegten Eingangszeilen die ursprüngliche und die neue Funktion vergleichen.

Gleichwertig ist nicht dasselbe wie gleich geschrieben

Die Reihenfolge der Terme ist egal. Auch unterschiedliche Termauswahlen können gleichwertig und gleich kurz sein. Vergleiche deshalb nicht nur die Zeichenfolge, sondern die Ausgangswerte. Bei X-Zeilen sind Unterschiede erlaubt. Eine vollständige Tabellenprüfung beweist die Funktion für die festgelegten Zeilen. Sie beweist allein noch nicht, dass keine kürzere Gleichung existiert. Dafür musst du die Abdeckungen vergleichen.

Grenzen des Verfahrens

K-Maps sind besonders übersichtlich für drei oder vier Variablen. „Minimal“ meint hier zunächst eine SOP mit möglichst wenigen Produkttermen, danach möglichst wenigen Literalen. Das garantiert nicht automatisch die schnellste oder stromsparendste physikalische Schaltung. Verzögerungen und mögliche Glitches werden durch die stabile Wahrheitstabelle nicht beschrieben.

07 / DER PYTHON-TEIL

Quine-McCluskey: gruppieren als Algorithmus

Das Verfahren sucht dieselben Vereinfachungen ohne gezeichnete Karte. Es arbeitet mit Bitmustern. Ein Strich steht für eine Variable, die innerhalb der Gruppe wechseln darf.

Muster	Kombinationen	Term
100	nur Index 4	$\overline{A}\overline{B}\overline{C}$
101	nur Index 5	$\overline{A}\overline{B}C$
10-	Indizes 4 und 5	$\overline{A}\overline{B}$

Der Ablauf in fünf Schritten

1. Starte mit den Bitmustern der Einerindizes und Don't-Cares.
2. Kombiniere zwei Muster, die bei gleichen Strichpositionen in genau einem weiteren Bit verschieden sind. Ersetze dieses Bit durch einen Strich.
3. Wiederhole das Kombinieren. Muster, die sich nicht weiter vergrößern lassen, sind Primimplikanten.
4. Baue eine Tabelle: Welcher Primimplikant deckt welche benötigten Einerindizes ab? X-Zellen müssen nicht abgedeckt werden.
5. Wähle eine vollständige Abdeckung mit möglichst wenigen Termen; bei Gleichstand mit möglichst wenigen Literalen.

Beispiel: 100 und 101 werden 10-. Dagegen lassen sich 100 und 111 nicht direkt verbinden: Sie unterscheiden sich an zwei Stellen.

Wichtig für deinen aktuellen Python-Stand

Ein vollständiger Minimierer ist deutlich größer als dein Tag-7-Tool. Deshalb liegen eine **Startdatei mit Teilaufgaben** und ein **ausführbarer Referenzprüfer** bereit. Löse die Karten zuerst selbst. Verwende den Prüfer anschließend zum Vergleichen und bearbeite den eigenen Algorithmus Schritt für Schritt.

```
python3 qm_referenz.py --fall 1
```

Der Referenzprüfer zeigt eine minimale SOP für den ausgewählten Fall. Der Vergleich aller festgelegten Tabellenzeilen bleibt Teil deiner Auswertung. Bei mehreren Minimalformen darf deine Formel anders aussehen.

Literatur und Umfang

Harris & Harris: *Digital Design and Computer Architecture, RISC-V Edition* (2022), 2.7, Buchseiten 73–81 (PDF 97–105). De Morgan/POS: 2.2–2.3. Die Quine-McCluskey-Umsetzung ist eine ergänzende Praxisaufgabe aus dem Kalender W02 T2, kein hier übernommenes Buchbeispiel. Die zehn Karten im Aufgabenblatt sind eigene Übungen im Themenumfang von Kapitel 2.7.